

ASCENDA

Frontend i18n Remediation Plan

Execution-Ready — 10-Day Sprint — April 2026

1. Executive Summary

Metric	Value
Current maturity	1.5 / 4 — partial keys, two disconnected language systems
Target maturity	4 / 4 — full EN/FR, AI-aligned, runtime language switching
Key architectural decision	PlanSettings.Language is the single source of truth for ALL language state
Total estimated effort	10 person-days (one developer)
Blocking items (ship-stopper)	2: AI language not propagated • vue-i18n locale not synced to PlanSettings.Language

Top 5 Blockers

#	Blocker	Impact	Fix
B 1	Language never sent to AI API (aiStore.ts)	AI narration always English regardless of plan language	30 min fix
B 2	vue-i18n locale set from env var, never updated at runtime	All t() strings ignore PlanSettings.Language	30 min fix
B 3	All sidebar category + AI feature labels hardcoded	Navigation always English	1 day
B 4	All scenario module names/descriptions hardcoded	Home screen always English	1 day
B 5	Enums (viability, risk, plan status) displayed raw from API	Untranslated codes visible to users	1 day

2. Target Architecture

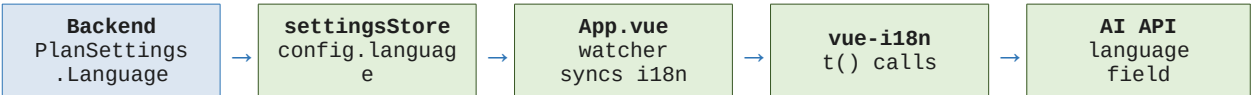
2.1 Single Source of Truth

PlanSettings.Language ('fr' | 'en') is the ONLY language source. It lives in the Pinia settingsStore under config.language and is populated when the user loads a plan.

Why VITE_DEFAULT_LOCALE must be removed from runtime logic: The env var is a deployment-time constant. It cannot differ per user, per plan, or per session. Using it as the vue-i18n locale means every French plan opened in an English deployment gets English labels. The env var may still be used as a build-time fallback during SSR preloading only — never as the runtime language authority.

Rule: If you are reading import.meta.env.VITE_DEFAULT_LOCALE outside of i18n.ts initialization, you are doing it wrong.

2.2 Language Flow — Final State



Step	Where	What happens	Rule
1	API response	PlanConfig.language ('fr' 'en') returned with plan data	Read once, never mutate
2	settingsStore	Stored as config.language via fetchConfig()	Single Pinia ref — no copies
3	App.vue	watch() syncs i18n.global.locale to config.language	One watcher, root component
4	vue-i18n	All t() calls resolve in active locale	Never call t() before locale is set
5	aiStore.ts	buildBaseContext() injects language from settingsStore	Every AI call, no exceptions
6	useDecimal	getLocale() maps language → IETF locale string	Already correct — no change needed

2.3 Vue i18n Synchronization — Exact Implementation

Place this watcher in App.vue, inside the <script setup> block, after imports. It must be at the root component level so it runs for the entire application lifetime.

App.vue — language watcher

```
// src/App.vue — add inside <script setup>
import { watch } from 'vue'
import { useI18n } from 'vue-i18n'
import { useSettingsStore } from '@/features/settings/stores/settingsStore'

const { locale } = useI18n()
const settingsStore = useSettingsStore()

watch(
  () => settingsStore.config?.language,
  (lang) => {
    if (lang) locale.value = lang // sync vue-i18n
  },
  { immediate: true } // run on first load
```

)

Lifecycle behaviour: immediate: true ensures the locale is set as soon as settingsStore is hydrated from the API, even before any component renders. If config is still null during the initial render, vue-i18n falls back to its configured fallbackLocale ('en'). Once the API responds and settingsStore.config.language is set, the watcher fires and all reactive t() bindings update.

Fallback behaviour: The i18n.ts fallbackLocale must remain 'fr' (the application default). If a key is missing in the active locale, vue-i18n silently falls back to 'fr'. Do NOT use VITE_DEFAULT_LOCALE as the fallback — hardcode 'fr'.

2.4 AI Integration Contract

NarrationContext (src/features/ai/types.ts) must declare a language field. buildBaseContext() must populate it. Every AI endpoint on the backend is expected to read this field and instruct its system prompt accordingly.

```
types.ts — add language to NarrationContext
// src/features/ai/types.ts — add to NarrationContext interface
export interface NarrationContext {
  plan_name: string
  scenario_name: string
  period_label: string
  currency: string
  user_role: NarrationUserRole
  language: 'fr' | 'en' // ← ADD THIS FIELD
  narration_type?: NarrationType
  // ... rest of fields unchanged
}
```

```
aiStore.ts — add language to buildBaseContext()
// src/features/ai/stores/aiStore.ts — update buildBaseContext()
function buildBaseContext(): NarrationContext {
  const planStore = usePlanStore()
  const scenarioStore = useScenarioStore()
  const reportStore = useReportStore()
  const settingsStore = useSettingsStore()
  const productStore = useProductStore()

  // ... existing metric derivation unchanged ...

  return {
    plan_name: plan?.name ?? 'Financial Plan',
    scenario_name: scenario?.name ?? 'Base Scenario',
    period_label: periodLabel,
    currency,
    user_role: resolveRole(),
    language: settingsStore.config?.language ?? 'fr', // ← ADD
    // ... rest of fields unchanged
  }
}
```

Backend contract: every AI handler (narrate, unit-economics, assumption-review, etc.) must read context.language and inject it into the system prompt as: "Respond exclusively in {language}. Do not mix languages."

3. Mandatory Coding Rules

These rules apply to ALL frontend developers from this point forward. They are non-negotiable for any PR that touches UI components.

RULE 1 — No hardcoded UI string

Every user-visible string **MUST** go through `t()`. The only exception is developer-only tooling (DEV badges, `console.log`). Rationale: a hardcoded string is impossible to translate without code changes. It also breaks the runtime language switch.

X BAD — never do this

```
// In a Vue template:  
<h2>Viability Score</h2>  
<Tag value="Strengths" />  
<button>New Scenario</button>
```

/ GOOD — correct pattern

```
// In a Vue template:  
<h2>{{ t('ai.viabilityScore') }}</h2>  
<Tag :value="t('ai.strengths')" />  
<button>{{ t('scenario.newScenario') }}</button>
```

RULE 2 — All enums use i18n mapping

Backend enum codes (e.g. 'critical', 'strong', 'draft') are **NEVER** displayed raw. They are always passed through `t()` with a namespaced key. Rationale: enum codes are language-neutral contracts between backend and frontend. Displaying them raw leaks implementation details and prevents translation.

X BAD — never do this

```
// Displaying raw urgency from API:  
<span>{{ risk.urgency }}</span>  
  
// Hardcoded map in component:  
const LABELS = {  
  critical: 'Critical',  
  high: 'High',  
}
```

/ GOOD — correct pattern

```
// Via i18n:  
<span>  
  {{ t(`enums.risk.urgency.${risk.urgency}`) }}  
</span>  
  
// Severity color from code (language-neutral):  
const SEVERITY = {  
  critical: 'danger', high: 'warning',  
}
```

RULE 3 — No raw backend value displayed to users

Any value that comes from the API and is displayed in the UI must either (a) be AI-generated text (display as-is, language is backend-controlled) or (b) be a code that is mapped through `t()`. There is no third option.

X BAD — never do this

```
// Plan status from API displayed raw:  
<Tag :value="plan.status" />  
// Output: "draft" or "approved"
```

/ GOOD — correct pattern

```
// Plan status mapped via i18n:  
<Tag  
  :value="t(`enums.planStatus.${plan.status}`)"  
  :severity="STATUS_SEVERITY[plan.status]"  
>  
// Output: "Brouillon" or "Approuvé"
```

RULE 4 — AI structural labels use i18n

AI-generated content (headlines, narratives, risk descriptions) is displayed verbatim — its language is controlled by the backend. BUT the structural labels around AI content (section headings, card titles, button labels) are frontend responsibility and MUST use `t()`. Mixing is the bug.

X BAD — never do this	✓ GOOD — correct pattern
<pre><!-- Section heading hardcoded, AI content dynamic --> <h3>Strengths</h3> <p>{{ analysis.highlights.strengths[0] }}</p> <!-- If plan is FR: heading=EN, content=FR --></pre>	<pre><!-- Both section heading and AI content correct --> <h3>{{ t('ai.strengths') }}</h3> <p>{{ analysis.highlights.strengths[0] }}</p> <!-- If plan is FR: heading=FR, content=FR ✓ --></pre>

RULE 5 — New components must be i18n-compliant from day one

Any new Vue component that contains user-visible text must import `useI18n` and use `t()` for all labels. PRs adding hardcoded strings to new files will be blocked. Template strings in props passed from parent views must also be i18n keys, not literals.

4. Component-Level Remediation

4.1 Navigation — AppSidebar.vue

The 8 category group labels and all 11 AI feature labels are hardcoded. Individual nav item labels already use `t()` correctly and need no changes.

Current hardcoded string	New i18n key	Namespace
'Data Input'	nav.dataInput	nav
'Financial Statements'	nav.financialStatements	nav
'Analysis'	nav.analysis	nav
'Cash & Budget'	nav.cashAndBudget	nav
'Dashboards'	nav.dashboards	nav
'AI Insights'	nav.aiInsights	nav
'Advanced'	nav.advanced	nav
'Version History'	nav.versionHistory	nav
'Narration'	nav.aiNarration	nav
'Unit Economics'	nav.aiUnitEconomics	nav
'Assumption Review'	nav.aiAssumptionReview	nav
'Benchmark Commentary'	nav.aiBenchmarkCommentary	nav
'Portfolio Mix'	nav.aiPortfolioMix	nav
'Driver Advisor'	nav.aiDriverAdvisor	nav
'Scenario Suggestion'	nav.aiScenarioSuggestion	nav
'Sensitivity Narrative'	nav.aiSensitivityNarrative	nav
'Investor Memo'	nav.aiInvestorMemo	nav
'Break-Even'	nav.breakEven	nav
'Cap Table'	nav.capTable	nav

Refactor pattern for all group labels:

X BAD — never do this	/ GOOD — correct pattern
<pre>{ label: 'Data Input', icon: 'pi pi-pencil', items: [...] }</pre>	<pre>{ label: t('nav.dataInput'), icon: 'pi pi-pencil', items: [...] }</pre>

For AI feature labels with lock suffix, compute the label reactively:

AppSidebar.vue — locked feature label

// Before:

```
label: isPro.value ? 'Unit Economics' : 'Unit Economics □'
```

// After:

```
label: isPro.value
  ? t('nav.aiUnitEconomics')
  : t('nav.aiUnitEconomics') + ' □'
```

4.2 Scenario Dashboard — Module Grid

ALL_MODULES is a static array of 17 objects. The name and desc fields must become i18n key references evaluated at render time, not string literals baked into the array. The mobileBlocked flag and other non-display properties are unchanged.

X BAD — never do this	/ GOOD — correct pattern
<pre>// ScenarioDashboardView.vue const ALL_MODULES = [{ name: 'P&L Statement', desc: 'Income statement with adjustments', icon: 'pi pi-chart-bar', path: '/pnl', mobileBlocked: false, }, // ...]</pre>	<pre>// ScenarioDashboardView.vue const { t } = useI18n() const ALL_MODULES = computed(() => [{ name: t('modules.pnl.name'), desc: t('modules.pnl.desc'), icon: 'pi pi-chart-bar', path: '/pnl', mobileBlocked: false, }, // ...])</pre>

Critical: ALL_MODULES must become a computed() so that it re-evaluates when the locale changes at runtime. A plain const array is evaluated once at component setup and will not update when the language switches.

Complete module key mapping:

Module	name key	desc key
Settings	modules.settings.name	modules.settings.desc
Products/Services	modules.products.name	modules.products.desc
Revenues	modules.revenues.name	modules.revenues.desc
Staff	modules.staff.name	modules.staff.desc
CapEx	modules.capex.name	modules.capex.desc
OpEx	modules.opex.name	modules.opex.desc
P&L Statement	modules.pnl.name	modules.pnl.desc
Financial Plan	modules.fiplan.name	modules.fiplan.desc
Functional P&L	modules.pnlCash.name	modules.pnlCash.desc
Balance Sheet	modules.bsheets.name	modules.bsheets.desc
Ratios	modules.ratios.name	modules.ratios.desc
WCR	modules.wcr.name	modules.wcr.desc
Cash Flow	modules.cash.name	modules.cash.desc
Budget	modules.budget.name	modules.budget.desc
Graphs	modules.graphs.name	modules.graphs.desc
Full Report	modules.report.name	modules.report.desc
Snapshots	modules.snapshots.name	modules.snapshots.desc

4.3 AI Components — ScenarioAnalysisCard & ScenarioIntelligenceSection

These components mix AI-generated narrative text (language controlled by backend) with hardcoded English structural labels. The fix is to replace every hardcoded structural label with t(). AI content (headlines, narrative paragraphs, strength/weakness/risk body text) is displayed verbatim as returned by the AI.

Complete key list for ScenarioAnalysisCard.vue:

Hardcoded string	i18n key	Type
'Viability Score'	ai.viabilityScore	Section heading
'Highlights'	ai.highlights	Section heading
'Strengths'	ai.strengths	Sub-heading
'Weaknesses'	ai.weaknesses	Sub-heading
'Risks'	ai.risks	Section heading
'Root Causes'	ai.rootCauses	Label
'Trend Insights'	ai.trendInsights	Section heading
'AI Commentary'	ai.commentary	Section heading
'(Fallback)'	ai.fallbackBadge	Badge label
'Refresh analysis'	ai.refreshAnalysis	Tooltip

Complete key list for ScenarioIntelligenceSection.vue:

Hardcoded string	i18n key	Type
'Scenario Intelligence'	ai.sectionTitle	Section header
'Refresh analysis'	ai.refreshAnalysis	Tooltip/button
'Strengths'	ai.strengths	Card heading
'Top Risks'	ai.topRisks	Card heading
'Unlock Full Analysis'	ai.unlockCta	Button label
'View Full Analysis'	ai.viewFullAnalysis	Button label

4.4 Financial Views

Each financial view requires extracting section headings, tab names, table column headers, and empty-state messages. The pattern is the same for all views. Below is the complete key plan per view.

View file	Keys to extract	Example key
PnlView.vue	Section: 'Manual Adjustments', 'P&L Report'; Tab: 'P&L Report', 'Graphs'; column headers (Line Item, Year N, % of Rev.); empty state	pnl.manualAdjustments
BSheetView.vue	Page title 'Balance Sheet'; tab labels; column headers (Line Item, Opening, Y1–Y5); section headings (Assets, Liabilities); chart titles	bsheet.pageTitle
RatiosView.vue	All 6 tab names (Sales & Margins, Operational KPIs, Profitability, Equity & Leverage, Valuation, Charts); all column headers; KPI labels	ratios.tabSalesMargins

View file	Keys to extract	Example key
WcrView.vue	Section headings (Customer WCR, Inventory WCR, Supplier WCR, Summary, Fiscal & Social); all table headers; tab names	wcr.customerSection
PnlCashView.vue	Tab names (Functional P&L, Graphs); section heading; note text; chart titles	pnlCash.tabFunctional
GraphDashboardView.vue	All 7 chart titles; info box text	graphs.salesAnalysis

Refactor pattern for DataTable column headers (applies to all views):

X BAD — never do this	✓ GOOD — correct pattern
<pre><Column field="label" header="Line Item" frozen /></pre>	<pre><Column field="label" :header="t('common.lineItem')" frozen /></pre>

Refactor pattern for tab labels:

X BAD — never do this	✓ GOOD — correct pattern
<pre><Tab value="functional"> Functional P&L </Tab></pre>	<pre><Tab value="functional"> {{ t('pnlCash.tabFunctional') }} </Tab></pre>

4.5 Shared Components

KFormLegend: This component accepts a description prop. The responsibility for translating this string lies with the PARENT, not the component. Each view that passes a description string to KFormLegend must pass a t() result instead of a literal.

X BAD — never do this	✓ GOOD — correct pattern
<pre><KFormLegend description="5-year projected balance sheet," description=" fully computed from your inputs." /></pre>	<pre><KFormLegend :description="t('bsheet.legendDescription')" /></pre>

KYearGrid: Row labels are built programmatically in the parent view as computed arrays. The label field in each row object must use t().

KYearGrid row labels
<pre>// PnlView.vue – row construction (simplified) const gridRows = computed(() => [{ field: 'capitalisedProduction', label: t('pnl.capitalisedProduction'), // ← t(), not a string literal editable: true, }, // ...])</pre>

DataTable (PrimeVue): Column headers passed via the header prop must be bound with : prefix so they are evaluated as expressions, not static strings. See example in 4.4 above.

Toast messages (useToast): All toast summary and detail strings must use `t()`. Group them under a `messages.*` namespace.

X BAD — never do this	✓ GOOD — correct pattern
<pre>toast.add({ severity: 'success', summary: 'Status updated', detail: `\${name} is now \${status}`, })</pre>	<pre>toast.add({ severity: 'success', summary: t('messages.statusUpdated'), detail: t('messages.statusDetail', { name, status: t('enums.planStatus.\$ {status}') })), })</pre>

5. Enum Mapping Strategy

Standard pattern: enum codes from the API are passed as dynamic key segments to `t()`. The colour/severity mapping uses a separate language-neutral constant object (never translated).

```
Enum mapping pattern
// Generic pattern for any enum:
t(`enums.{namespace}.{code}`)

// Viability status:
t(`enums.viability.${analysis.viabilityStatus}`)

// Risk urgency:
t(`enums.risk.urgency.${risk.urgency}`)

// Plan status:
t(`enums.planStatus.${plan.status}`)
```

Complete enum mapping table — all enums requiring i18n treatment:

Enum	Backend codes	i18n namespace	Colour map (language-neutral)
Viability status	strong • viable • at_risk • critical	enums.viability.*	VIABILITY_SEVERITY = { strong:'success', viable:'info', at_risk:'warning', critical:'danger' }
Risk urgency	critical • high • medium • low	enums.risk.urgency.*	URGENCY_COLOR = { critical:'#C00000', high:'#E97000', medium:'#2E75B6', low:'#595959' }
Risk severity	immediate • long_term	enums.risk.severity.*	n/a
Plan status	draft • review • approved • archived	enums.planStatus.*	STATUS_SEVERITY = { draft:'secondary', review:'warning', approved:'success', archived:'contrast' }
Tier	free • pro • enterprise	enums.tier.*	TIER_COLOR = { free:'secondary', pro:'info', enterprise:'warn' }
Break-even status	profitable • break_even • at_risk • loss	enums.breakEven.*	BEP_SEVERITY = { profitable:'success', break_even:'info', at_risk:'warning', loss:'danger' }

Implementation example for viability badge — separating label (i18n) from colour (code-driven):

```
Viability badge — correct pattern
// ScenarioAnalysisCard.vue
const { t } = useI18n()

const VIABILITY_SEVERITY: Record<string, string> = {
  strong: 'success', viable: 'info',
  at_risk: 'warning', critical: 'danger'
}

// In template:
<Tag
  :value="t(`enums.viability.${analysis.viabilityStatus}`)"
  :severity="VIABILITY_SEVERITY[analysis.viabilityStatus] ?? 'secondary'"
/>
```

6. Target File Structure

6.1 Directory Layout

```
src/locales/ structure
src/
  locales/
    fr.json      ← primary language (application default)
    en.json      ← secondary language
  plugins/
    i18n.ts      ← createI18n, load from locales/, NO inline messages
```

6.2 Updated i18n.ts

```
i18n.ts — final implementation
// src/plugins/i18n.ts
import { createI18n } from 'vue-i18n'
import fr from '../locales/fr.json'
import en from '../locales/en.json'

export const i18n = createI18n({
  legacy: false,
  locale: 'fr', // ALWAYS 'fr' – overridden at runtime
  fallbackLocale: 'fr', // fallback = primary language
  messages: { fr, en },
  // Remove VITE_DEFAULT_LOCALE from here
})
```

6.3 fr.json Namespace Structure

The following shows the complete namespace skeleton. Each namespace maps to a distinct concern. Translators work on individual JSON files without needing to understand the codebase.

```
fr.json — complete namespace skeleton
{
  "common": {
    "save": "Enregistrer", "cancel": "Annuler", "delete": "Supprimer",
    "loading": "Chargement...", "noData": "Aucune donnée",
    "lineItem": "Intitulé", "year": "Année", "total": "Total",
    "actions": "Actions", "search": "Rechercher"
  },
  "nav": {
    // existing keys unchanged +
    "dataInput": "Saisie des données",
    "financialStatements": "États financiers",
    "analysis": "Analyse",
    "cashAndBudget": "Trésorerie & Budget",
    "dashboards": "Tableaux de bord",
    "aiInsights": "Analyse IA",
    "advanced": "Avancé",
    "versionHistory": "Historique des versions",
    "aiNarration": "Narration",
    "aiUnitEconomics": "Unit Economics",
    "aiAssumptionReview": "Révision des hypothèses",
    "aiBenchmarkCommentary": "Commentaire benchmark",
    "aiPortfolioMix": "Mix produits",
    "aiDriverAdvisor": "Conseil métrique",
    "aiScenarioSuggestion": "Suggestion de scénario",
    "aiSensitivityNarrative": "Analyse de sensibilité",
    "aiInvestorMemo": "Mémo investisseur",
    "breakEven": "Seuil de rentabilité",
```

```

    "capTable": "Tableau de capitalisation"
  },
  "modules": {
    "pnl": { "name": "Compte de résultat", "desc": "Résultat avec ajustements manuels"
  },
    "bsheet": { "name": "Bilan", "desc": "Actif, passif, capitaux propres" },
    "ratios": { "name": "Ratios", "desc": "KPIs financiers" },
    "wcr": { "name": "BFR", "desc": "Besoins en fonds de roulement" },
    "pnlCash": { "name": "P&L Fonctionnel", "desc": "P&L Anglo-Saxon par fonction" },
    "graphs": { "name": "Graphiques", "desc": "Graphiques financiers annuels" },
    "settings": { "name": "Paramètres", "desc": "Configuration, bilan d'ouverture,
BFR" },
    "products": { "name": "Produits / Services", "desc": "Modèles de revenus et prix" },
    "revenues": { "name": "Revenus", "desc": "Pilotes et prévisions de
revenus" },
    "staff": { "name": "Personnel", "desc": "Effectifs, salaires, primes" },
    "capex": { "name": "Investissements", "desc": "Immobilisations et amortissements"
  },
    "opex": { "name": "Charges", "desc": "Charges d'exploitation par
catégorie" },
    "fiplan": { "name": "Plan de financement", "desc": "Structure de financement" },
    "cash": { "name": "Trésorerie", "desc": "Tableau de flux de trésorerie
mensuel" },
    "budget": { "name": "Budget", "desc": "Comparaison réel vs budget" },
    "report": { "name": "Rapport complet", "desc": "Export en document Word" },
    "snapshots": { "name": "Versions", "desc": "Historique des versions" }
  },
  "ai": {
    "viabilityScore": "Score de viabilité",
    "highlights": "Points clés",
    "strengths": "Points forts",
    "weaknesses": "Points de vigilance",
    "risks": "Risques",
    "rootCauses": "Causes racines",
    "trendInsights": "Tendances",
    "commentary": "Commentaire IA",
    "fallbackBadge": "(Secours)",
    "refreshAnalysis": "Rafraîchir l'analyse",
    "sectionTitle": "Intelligence scénario",
    "topRisks": "Risques principaux",
    "unlockCta": "Activer l'analyse complète",
    "viewFullAnalysis": "Voir l'analyse complète"
  },
  "enums": {
    "viability": {
      "strong": "Solide", "viable": "Viable",
      "at_risk": "Fragile", "critical": "Critique"
    },
    "risk": {
      "urgency": {
        "critical": "Critique", "high": "Élevé",
        "medium": "Modéré", "low": "Faible"
      },
      "severity": {
        "immediate": "Immédiat", "long_term": "Long terme"
      }
    },
    "planStatus": {
      "draft": "Brouillon", "review": "En révision",
      "approved": "Approuvé", "archived": "Archivé"
    },
    "tier": {
      "free": "Gratuit", "pro": "Pro", "enterprise": "Entreprise"
    },
    "breakEven": {
      "profitable": "Rentable", "break_even": "À l'équilibre",
      "at_risk": "Fragile", "loss": "Déficitaire"
    }
  }

```

```
}
},
"messages": {
  "statusUpdated": "Statut mis à jour",
  "statusDetail": "\"{name}\" est maintenant \"{status}\"",
  "error": "Erreur",
  "updateFailed": "Impossible de mettre à jour le statut.",
  "noPlansYet": "Aucun plan. Créez votre premier plan d'affaires !",
  "fullTabletOnTablet": "Tableau complet disponible sur tablette et bureau."
},
"pnl": {
  "pageTitle": "Compte de résultat",
  "tabReport": "Compte de résultat", "tabGraphs": "Graphiques",
  "manualAdjustments": "Ajustements manuels",
  "legendDescription": "Lignes optionnelles hors opérations courantes. Laisser à zéro si
non applicable."
},
"bsheet": {
  "pageTitle": "Bilan",
  "tabTable": "Tableau", "tabCharts": "Graphiques",
  "assets": "Actif", "liabilities": "Passif",
  "legendDescription": "Bilan prévisionnel sur 5 ans, entièrement calculé depuis vos
données."
},
"ratios": {
  "tabSalesMargins": "Ventes & Marges",
  "tabOperational": "KPIs opérationnels",
  "tabProfitability": "Rentabilité",
  "tabEquityLeverage": "Capitaux & Levier",
  "tabValuation": "Évaluation",
  "tabCharts": "Graphiques"
},
"wcr": {
  "customerSection": "BFR Clients",
  "inventorySection": "BFR Stocks",
  "supplierSection": "BFR Fournisseurs",
  "summarySection": "Synthèse",
  "fiscalSection": "Fiscal & Social"
},
"scenario": {
  "newScenario": "Nouveau scénario",
  "wizard": "Assistant de saisie"
}
}
```

7. Migration Plan

10 phases across 10 working days. Each phase is independent and can be PR-reviewed separately. Start each phase only after the previous one is merged to avoid conflicts.

Phase 1 Day 1

Immediate Fixes — Language Sync & AI

- **Files:** `src/features/ai/types.ts`, `src/features/ai/stores/aiStore.ts`, `src/App.vue`, `src/plugins/i18n.ts`
- Add language: 'fr' | 'en' field to NarrationContext interface in types.ts
- In aiStore.ts buildBaseContext(): add language: settingsStore.config?.language ?? 'fr'
- In App.vue <script setup>: add watch() to sync i18n locale to settingsStore.config.language with immediate: true
- In i18n.ts: change locale initialization to hardcoded 'fr' — remove VITE_DEFAULT_LOCALE from runtime locale

Validation criterion	How to verify
AI requests include language field	Open browser DevTools Network tab, trigger narration, confirm JSON body has "language": "fr"
AI response is in French for FR plan	Load a plan with language='fr', open Narration, verify French text returned
vue-i18n locale updates when plan loads	Add console.log in watcher, verify it fires with correct locale on plan load
No regression on existing t() calls	Run existing E2E suite, check sidebar labels still render

Phase 2 Day 2

Navigation — AppSidebar

- **Files:** `src/components/layout/AppSidebar.vue`, `src/locales/fr.json`, `src/locales/en.json`
- Create `src/locales/` directory with `fr.json` and `en.json` (migrate from inline i18n.ts messages)
- Add all nav.* keys listed in section 4.1 to both locale files
- Replace all 19 hardcoded category + AI feature labels in AppSidebar.vue with t() calls
- Update i18n.ts to import from JSON files instead of inline objects

Validation criterion	How to verify
All sidebar category labels render from translations	Switch locale to 'en', verify labels change to English
Locked feature labels still append lock icon	Verify Unit Economics shows "□" suffix when on free tier
No missing key warnings in browser console	Check console for [vue-i18n] Missing translation key warnings

Phase 3 Day 3

Scenario Dashboard — Module Grid

- **Files:** `src/features/scenarios/views/ScenarioDashboardView.vue`, `src/locales/fr.json`, `src/locales/en.json`
- Add all `modules.*.name` and `modules.*.desc` keys (17 modules × 2 = 34 keys) to both locale files
- Convert `ALL_MODULES` static array to `ALL_MODULES computed()` => [...] using `t()`
- Verify modules computed re-evaluates when locale changes
- Update template: replace any remaining hardcoded strings ('Wizard' button → `t('scenario.wizard')`)

Validation criterion	How to verify
Module cards render French names for FR plan	Load FR plan, verify 'Compte de résultat' shown (not 'P&L Statement')
Module filter (mobile) still works	Test mobile view, verify 6 modules still visible and correctly named
Switching to EN plan shows English module names	Load EN plan, verify English names

Phase 4 Day 4

AI Components

- **Files:** `src/features/ai/components/ScenarioAnalysisCard.vue`, `src/features/scenarios/components/ScenarioIntelligenceSection.vue`, `locales/`
- Add all `ai.*` keys (13 keys) to both locale files (see complete list in section 4.3)
- Replace all hardcoded labels in `ScenarioAnalysisCard.vue` with `t()` calls
- Replace all hardcoded labels in `ScenarioIntelligenceSection.vue` with `t()` calls
- Verify AI-generated content (body text) is NOT passed through `t()` — display verbatim

Validation criterion	How to verify
Section headings render in French for FR plan	'Points clés' shown instead of 'Highlights'
AI narrative content is NOT double-translated	French AI text still displays as returned by API
No mixed-language page	All structural labels and AI content are in same language

Phase 5 Day 5

Enum Mapping

- **Files:** `ScenarioAnalysisCard.vue`, `ScenarioIntelligenceSection.vue`, `DashboardView.vue`, `AppTopbar.vue`, `locales/`
- Add `enums.*` namespace to both locale files (6 enums, ~20 total keys — see section 5)
- Replace `t('enums.viability.${status}')` throughout AI components
- Replace `t('enums.risk.urgency.${urgency}')` in `ScenarioAnalysisCard`
- Replace hardcoded plan status options in `DashboardView` (Draft/Review/Approved/Archived)
- Replace tier labels in `AppTopbar` (Pro/Enterprise/Free)
- Add `SEVERITY/COLOR` constant maps (language-neutral) for all badge components

Validation criterion	How to verify
Viability shows 'Solide' for FR plan with strong status	Load FR plan with strong viability score
Risk urgency shows 'Critique' for FR plan	Verify risk badges in ScenarioAnalysisCard
Plan status dropdown shows 'Brouillon' etc. in FR	Open plan status selector in DashboardView
Badge colours are unchanged	Verify green/red/orange/yellow remain correct for all statuses

Phase 6

Day 6

Financial Views — P&L & Balance Sheet

- **Files:** `src/features/pnl/views/PnlView.vue`, `src/features/bsheet/views/BSheetView.vue`, `locales/`
- Add `pnl.*` and `bsheet.*` namespaces to both locale files
- Replace section headings, tab names, DataTable column headers with `t()` in both views
- Replace `KFormLegend` description string props with `:description="t('pnl.legendDescription')"`
- Replace row label strings in `KYearGrid` row definitions with `t()` calls

Validation criterion	How to verify
P&L tab labels in French for FR plan	'Compte de résultat' and 'Graphiques' tabs visible
Balance sheet column headers in French	Verify 'Actif', 'Passif' headings in detail table
KFormLegend description is French	Verify explanation text below table headers is in French

Phase 7

Day 7

Financial Views — Ratios, WCR, Graphs, PnlCash

- **Files:** `RatiosView.vue`, `WcrView.vue`, `GraphDashboardView.vue`, `PnlCashView.vue`, `locales/`
- Add `ratios.*`, `wcr.*` to both locale files
- Replace all tab names, section headings, DataTable headers in all 4 files
- Replace chart title strings with `t()` in `GraphDashboardView` and other views

Validation criterion	How to verify
Ratios tabs render in French for FR plan	'Ventes & Marges' instead of 'Sales & Margins'
WCR section headings in French	'BFR Clients' instead of 'Customer WCR'
All 7 chart titles in French	Verify <code>GraphDashboardView</code> card headings

Phase 8

Day 8

Messages, Toasts & Dialogs

- **Files:** `DashboardView.vue`, `PlanOverviewView.vue`, all views using `useToast`, `locales/`
- Add `messages.*` namespace to both locale files
- Replace all `toast.add()` summary and detail strings with `t()`

- Replace confirmation dialog texts with t()
- Replace empty state messages across all views with t()
- Replace button labels (New Scenario, Lock, Unlock, Archive) with t()

Validation criterion	How to verify
Toast shows French text for FR plan	Trigger a status update on FR plan, verify French toast
Empty state message in French	Clear all plans and verify French empty state
Lifecycle buttons in French	Open Plan Overview, verify 'Verrouiller', 'Archiver' labels

Phase 9

Day 9

Shared Components & Cleanup

- **Files: All views passing string literals to KFormLegend, KYearGrid, DataTable column headers**
- Audit all remaining string literals with eslint-plugin-i18n check (see section bonus)
- Fix any remaining prop-passed strings in shared components
- Add explicit locale to all toLocaleDateString() and toLocaleString() calls
- Remove VITE_DEFAULT_LOCALE from any remaining runtime usages

Validation criterion	How to verify
ESLint i18n rule reports zero violations	Run <code>npx eslint src --ext .vue,.ts</code>
Date formatting consistent across views	Verify audit trail dates show correct FR format (dd/mm/yyyy)

Phase 10

Day 10

Validation, Testing & Sign-off

- **Full regression test of EN and FR modes**
- Playwright E2E: add language-switch test (see section 9)
- Run CI missing-key check script (see bonus section)
- Visual review: check for FR text overflow in compact components
- Final review against Definition of Done checklist (section 10)

8. Risk Analysis

Risk	Probability	Impact	Mitigation
Mixed-language screen during partial migration	HIGH	HIGH	Complete each phase in full before merging. Never commit a half-translated component. Use feature flags if needed.
ALL_MODULES as computed breaks dependent tests	MEDIUM	MEDIUM	Update tests to wrap in withSetup() with i18n plugin. Add t() mock in vitest setup.
FR text overflow in compact components (sidebar, KPI cards)	HIGH	MEDIUM	Add max-w + truncate class to sidebar labels. Add title attribute for full text on hover. Test each component in FR after Phase 2.
Missing translation key (blank UI)	MEDIUM	HIGH	Add CI check (see bonus). Set fallbackLocale: 'fr' so missing EN keys fall back gracefully.
AI response still in English after backend contract not updated	MEDIUM	HIGH	Confirm with backend team that each AI handler reads context.language before merging Phase 1.
VITE_DEFAULT_LOCALE still used somewhere at runtime	LOW	MEDIUM	Add grep check in CI: grep -r "VITE_DEFAULT_LOCALE" src/ should return zero results after Phase 9.

9. Testing Strategy

9.1 Unit Tests (Vitest)

For every component that uses `t()`, tests must mount with the `i18n` plugin. Add a shared test helper:

```
Vitest i18n helper
// src/test/i18nHelper.ts
import { createI18n } from 'vue-i18n'
import fr from '../locales/fr.json'
import en from '../locales/en.json'

export function createTestI18n(locale: 'fr' | 'en' = 'fr') {
  return createI18n({
    legacy: false, locale, fallbackLocale: 'fr',
    messages: { fr, en }
  })
}

// Usage in component test:
const wrapper = mount(ScenarioAnalysisCard, {
  global: { plugins: [createTestI18n('fr')] },
  props: { analysis: mockAnalysis },
})
expect(wrapper.text()).toContain('Points clés') // French label
```

9.2 E2E Tests (Playwright)

Add a language-switch spec to `e2e/i18n.spec.ts`:

```
e2e/i18n.spec.ts
// e2e/i18n.spec.ts
test('French plan shows French labels', async ({ page }) => {
  // Assume mock returns plan with language: 'fr'
  await page.goto('/plans/123/scenarios/456/modules')
  await page.waitForSelector('[data-testid="module-grid"]')
  // Module names must be French
  await expect(page.getByText('Compte de résultat')).toBeVisible()
  await expect(page.getByText('P&L Statement')).not.toBeVisible()
})

test('AI narration request includes language field', async ({ page }) => {
  const [request] = await Promise.all([
    page.waitForRequest(r => r.url().includes('/api/v1/') && r.url().includes('/narrate')),
    page.click('[data-testid="trigger-narration"]'),
  ])
  const body = JSON.parse(request.postData() ?? '{}')
  expect(body.context.language).toBe('fr')
})

test('English plan shows English labels', async ({ page }) => {
  // Mock returns plan with language: 'en'
  await page.goto('/plans/789/scenarios/012/modules')
  await expect(page.getByText('P&L Statement')).toBeVisible()
})
```

9.3 Visual / Layout Regression

- After Phase 2: screenshot sidebar in FR mode, verify no label overflow
- After Phase 3: screenshot module grid in FR mode, verify card descriptions fit

- After Phase 4: screenshot ScenarioAnalysisCard in FR mode with AI content, verify no mixed-language section
- After Phase 6–7: screenshot each financial view in FR mode at 375px (mobile) and 1280px (desktop)

9.4 Regression Checks

Check	Tool	Expected result
No hardcoded strings in Vue templates	ESLint (see bonus)	0 violations
No missing translation keys	CI script (see bonus)	0 missing keys
All existing Playwright tests pass	npm run test:e2e	0 failures
No <code>VITE_DEFAULT_LOCALE</code> in <code>src/</code>	grep -r in CI	0 matches
No console i18n warnings	Browser console during test	0 [vue-i18n] warnings

10. Definition of Done

The i18n remediation is complete when ALL of the following are true:

#	Criterion	How to verify
1	AI API calls include language field in every request	Network tab: all AI requests have context.language
2	AI narration returns French text for FR plan	Open Narration on FR plan, verify French response
3	vue-i18n locale syncs to PlanSettings.Language on plan load	Console log in watcher fires with correct locale
4	No user-visible string is hardcoded in any .vue or .ts file	ESLint i18n rule: 0 violations
5	All enum values are mapped through t()	No raw codes ('draft', 'critical', 'strong') visible in any rendered UI
6	Switching between EN and FR plans updates all UI labels	Load FR plan then EN plan consecutively — all labels change
7	No mixed-language screens (FR AI content + EN labels)	Visual review of all AI component pages
8	All FR text fits in its container without overflow	Visual regression at 375px and 1280px widths
9	All translation keys present in both fr.json and en.json	CI missing-key check: 0 missing keys
10	All existing Playwright tests pass	npm run test:e2e: 0 failures
11	VITE_DEFAULT_LOCALE not used at runtime	grep -r "VITE_DEFAULT_LOCALE" src/: 0 matches
12	Report generation unaffected (reportLabels.ts unchanged)	Generate Word report for FR and EN plans

